The cache is cheaper than the weights: measured bit costs and device-dependent capacity for LLM inference on 8 GB devices

Yibo Zhang[a,*], Muhammad Syafiq bin Mohd Pozi[a], Osman B Ghazali[a]

[a]*School of Computing, Universiti Utara Malaysia, Sintok, Kedah, Malaysia*

Abstract

Quantisation fits a large language model into an 8 GB device, and the literature optimises it for speed. We measure four mismatches between that literature and the device. *First*, the serving stack sets the budget before any bit does: grouped-query attention silently disables every fused attention kernel on our stack, the fallback materialises an $O(L^2)$ score matrix, and materialising the KV heads — a mathematically transparent rewrite — cuts attention peak memory 32× and 4k prefill time 5.8×. *Second*, 4-bit weights buy little or no time, because dequantisation is not fused into the matmul; quantisation here buys capacity, not throughput, so the objective must be capacity. *Third*, the per-bit cost runs against practice: with the published GPTQ for weights and KIVI's own quantiser for the cache, weights cost 11.39% perplexity where the cache costs 1.27% on Qwen3-0.6B, and 3.20% against 0.24% on Qwen3-1.7B; the ordering reproduces on WikiText-2, and a 2-bit passkey experiment shows the scheme, not the width alone, decides whether long-context retrieval survives. *Fourth*, capacity is device-dependent: usable memory differs 3.15× between a discrete-GPU laptop and a unified-memory edge module. We solve the allocation exactly and report both devices' allocations and the module's measured ceiling.

Keywords: quantisation, LLM inference, KV cache, edge inference, memory budget, bit allocation, energy efficiency

Highlights

- On an 8 GB device the serving stack, not the bit width, sets the memory budget.
- One KV-cache bit costs 9–13x less accuracy than one weight bit, on two models.
- 2-bit cache quantisation separates schemes where 4-bit does not: 5/5 against 0/5.
- The measured ceiling of the edge module is 3k tokens, not the 67k a model predicts.
- The optimal policy compresses the cache and leaves the weights at 8 bits.

CRediT authorship contribution statement

Yibo Zhang: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing - original draft, Writing - review & editing, Visualization. **Muhammad Syafiq bin Mohd Pozi:** Supervision. **Osman B Ghazali:** Supervision.

*Corresponding author.
Email address: zhang_yibo@ahsgs.uum.edu.my (Yibo Zhang)

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

All measurement scripts, the vendored third-party quantisers, the measurement files behind every table, and the scripts that regenerate the figures are publicly available at https://github.com/BRAD-cording/llm-8gb-bit-costs. A single command rebuilds the results and runs the checks described in Section 7.

1. Introduction

Deploying a large language model on an 8 GB device is no longer a research curiosity; it is the default target for on-device assistants, and the entire post-training quantisation (PTQ) literature is organised around it.

What kind of paper this is. It is a measurement paper. Its subject is what an 8 GB device actually does with the bits it is given, and its contributions are four measurements that contradict what the allocation literature assumes, plus the allocator that follows from them. We state that plainly because the opposite reading is available: the allocation step is a multiple-choice knapsack solved by

exact dynamic programming, which is textbook. Nothing here should be read as a new optimiser. What is new is the input to the optimiser — per-device usable memory, a measured workspace curve, and per-bit accuracy costs measured on both sides of the same model — and the finding that the objective has to be capacity rather than latency. Section 4 returns to this distinction. That literature optimises *weights*: rotations to flatten activations [1, 2, 3, 4], Hessian-aware rounding [5, 6], and mixed-precision schemes that spend more bits on sensitive layers [7, 8, 9, 10, 11]. System-level work co-designs the kernel with the format [12, 13, 14, 15, 16], and low-rank compensation has been used to recover what rounding loses [17, 18, 19]. The KV cache is either left in 16 bits or compressed by a separate line of work [20, 21, 22, 23, 24, 25, 26] that takes the weight precision as given. Serving stacks that manage the cache as a first-class resource [27, 27] do not choose weights against cache either.

At the context lengths that make these models useful, that division is the wrong one. From the published model cards this is arithmetic rather than a hazard: a KV cache costs 112 KiB per token for Qwen3-0.6B and 1.7B and 144 KiB for 4B and 8B, so at 32k tokens the cache is 3.76–4.83 GB against 0.22–4.09 GB of 4-bit weights, and the two are equal at 1,919 tokens for 0.6B, 6,144 for 1.7B, 12,319 for 4B and 21,388 for 8B. For an 8 GB device the cache overtakes the weights long before the context lengths these models are deployed at.

Once the cache is a first-class consumer, the problem becomes an allocation problem: given a fixed byte budget, how many bits for weights and how many for the cache? This paper argues that the answer is *device-dependent*, and that the dependence has a cause that can be measured but that no PTQ paper reports.

The cause is the gap between nominal and usable memory. Both of our devices are advertised as 8 GB. Measured on the day of the experiment, a discrete-GPU laptop has 6.93 GiB free for the model ($\rho = 0.133$: driver context, compositor) while a unified-memory Jetson Orin Nano Super has 4.32 GiB ($\rho = 0.419$: L4T, desktop, carve-out). The usable budgets differ by 1.6×. Because the cache grows linearly in context length while the weights do not, that difference changes the exchange rate between a weight bit and a token of context: dropping Qwen3-4B from 8-bit to 4-bit weights multiplies the attainable context by 1.57× on the laptop but by 5.76× on the Jetson — a 3.7× difference in the price of the same bit (Section 3). A single published configuration cannot be optimal on both.

Contributions.

1. **What sets the budget is measurable, and it is not the bit-width.** A silent attention-dispatch failure costs 32× in attention peak memory and 5.8× in 4k prefill time, and removing it extends the attainable context 4× before any quantisation decision is made; separately, the usable-memory fraction differs 3.15× between two nominally identical 8 GB targets. Both are properties of the stack and the device that the allocation literature treats as fixed. We release a model-free reproduction of the dispatch failure, which leaves no trace in the framework's output, and we report the edge module's *measured* ceiling beside the cost model's prediction for it (Sections 6.1 and 3).
2. **The per-bit cost of the two sides runs against practice.** With published implementations on both sides — the official GPTQ for weights, KIVI's own quantiser for the cache — one cache bit costs far less accuracy than one weight bit: 9.0× at 0.6B and 13.3× at 1.7B, and the ordering reproduces on WikiText-2. A 2-bit passkey experiment locates where the claim stops: at 4 bits the quantisation scheme does not change long-context retrieval, at 2 bits it decides it (Section 6.4).
3. **The consequence is an allocation problem whose objective must be capacity.** We formulate joint weight–KV allocation as a multiple-choice knapsack over measured per-device curves and solve it exactly, so the optimisation is textbook — the contribution is the inputs it runs on and the finding that latency cannot be the objective on this stack. The resulting policy compresses the cache and leaves the weights at 8 bits on both devices, and we report it against a deployed control rather than a modelled one (Sections 4–6.3).

What this paper does not claim. We do not claim a new quantiser: we use existing ones and allocate their output. We do not claim that the optimal allocation reverses between prefill and decode regimes — we tested that hypothesis and could not support it; the reasoning and the falsification are documented in the artifact (Appendix Appendix A). And we do not claim exhaustive literature coverage.

2. Related Work

2.1. Quantisation for speed

The dominant framing in post-training quantisation is that fewer bits mean faster inference. Rotations flatten activations so a single 4-bit quantiser suffices [1, 2, 3, 28, 29, 30]; second-order methods cancel rounding error [5, 31]; system papers report speedups from weight-only formats [32]. That framing is what our Section 6.2 measures directly, and on our stack it does not hold: the arithmetic stays fp16, so the only quantity that changes is bytes stored.

2.2. Capacity-oriented compression

A second line treats memory, not time, as the scarce resource: KV cache quantisation [20, 21, 22], joint memory–accuracy frontiers [23], and precision allocation under a storage budget [7, 8, 9]. Closest to our setting, [10] scores precision choices separately for prefill and decode and [33]

specialises formats per phase, while [16] keeps decode in high precision.

Reference [23] deserves a precise comparison because it is the nearest prior art. It maps a joint frontier over cache bit-width, cache granularity, chunked prefill and weight quantisation, and reports Pareto-optimal configurations at 68–78% memory reduction. Its weight side is fixed at 4-bit AWQ, however, so the weight/cache *split* is not among its decision variables; its frontier is indexed by memory consumed rather than by a per-bit accuracy cost measured end to end; and it reports no usable-memory fraction for its hardware, so the frontier cannot be re-targeted to a device with a different ρ. Our claim is narrower and different: that the per-bit cost of the two sides must be measured on the same footing, that the resulting optimum is device-dependent through ρ, and that on our stack the answer is to compress the cache and leave the weights alone.

None of these treats the weight/KV split as a decision variable under a measured device budget, and none reports the usable-memory fraction of the target device, which is the quantity that dominates our results.

2.3. Edge measurement and reporting

A recurring criticism is that quantisation results are not reproducible because the execution stack is unreported. Our measurements give that criticism a number: the same model on the same card supports 4k or beyond 16k context depending only on which attention kernel the framework selects (Section 6.1). This is also why we record the software stack and the device's memory state with every latency number, and why our ρ values are reported as a measurement procedure rather than as a constant. What we do *not* control is die temperature and clock throttling: our sweeps are short and configurations are interleaved, which bounds thermal ordering effects, but we did not instrument the clocks, so a throttling explanation for a small difference cannot be excluded from our data alone.

3. Usable Memory and the Weight-Bit Exchange Rate

What "8 GB" actually means. We measure the fraction of nominal memory that is unavailable to the model on the day of the experiment, rather than trusting the datasheet. On the laptop this is the CUDA context plus the compositor, read from the allocator; on the Jetson it is L4T, the desktop and the carve-out, read from the kernel. The difference is large enough to change conclusions:

The usable budgets differ by 1.6× although both devices are sold as 8 GB. ρ is not a constant: it moves with the OS image, with background load, and — on the Jetson — with the power mode, which we set once (25 W) and did not vary, so the power-mode sensitivity is a stated caveat rather than a measurement. This is why we report the procedure alongside the number. Any paper that states a device and a bit-width without stating ρ is reporting a configuration that does not transfer.

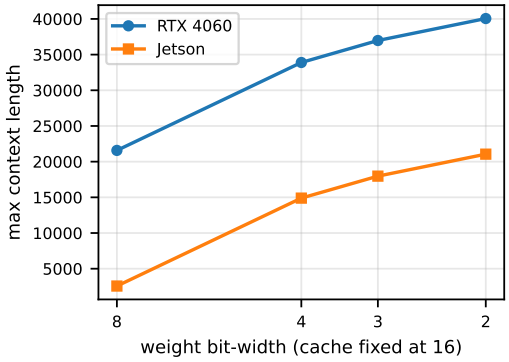


Figure 1: Attainable context versus weight width for Qwen3-4B with a 16-bit cache, from measured ρ and workspace. The two curves have different slopes, which is the whole reason a single published configuration cannot be optimal on both devices.

The exchange rate between a weight bit and a token. Because the cache grows linearly in context and the weights do not, a saved weight bit converts into context at a rate that depends on how tight the budget is. Figure 1 shows the conversion for Qwen3-4B with the cache held at 16 bits: dropping weights from 8 to 4 bits multiplies the attainable context by 1.57× on the laptop and by 5.76× on the Jetson, so the same bit is worth 3.7× more on the constrained device. The rates are computed from the measured ρ and workspace curves of Section 4, not from datasheet numbers.

Why this is a capacity argument, not a speed argument. It would be natural to read the curves above as "quantise the weights to go faster". They do not say that: Section 6.2 shows that the 4-bit path is in fact slower on this stack. What the curves say is that on a device this constrained, precision is the knob that decides how much context you can address at all, and that the exchange rate for that knob is device-dependent. That is the sense in which the allocation problem is worth solving.

4. Method

Why the objective is capacity, not latency. The natural formulation is to minimise serving time subject to an accuracy constraint. We tried that first and it is degenerate on this stack: latency is not a function of the bit-widths at all, since a 4-bit weight is still multiplied at fp16 precision (Table 3). An objective whose gradient with respect to the decision variables is zero cannot rank allocations. What bit-width *does* determine here is how much context fits, so that is what we optimise: maximise the attainable context subject to a bounded accuracy loss. This is a measured choice, not a modelling convenience, and it is the reason Table 3 appears before the method rather than after it.

Budget constraint. Let M be nominal memory and ρ the measured unusable fraction. An allocation $\mathbf{b}$ is feasible for a context length L iff

$$\sum_l \left(b_l^W \frac{|W_l|}{8} + s_l \right) + 2LdH_{kv}\frac{\bar{b}^{KV}}{8} \;\le\; M(1-\rho) - W_{\text{act}}(L), \tag{1}$$

where s_l are per-group scales and the cache term is linear in L. The workspace term is the one prior budget model gets wrong: it is neither zero nor constant. Measured, it is linear in the tokens of a single forward pass with a *model-dependent* coefficient and intercept: 0.142 MB per token and 9 MB for the 0.6B model, 0.217 and 161 MB for the 4B. The coefficient has to be refitted per model — using the 4B value for a 0.6B model over-reserves workspace by about 60% and so under-reports attainable context, a mistake we made and corrected. What it does not depend on is the device: the 0.6B curve fitted on the laptop predicts the Jetson's measured workspace to within 2% (Section 5). Because the term scales with the tokens in a single forward pass rather than with the conversation, chunked prefill bounds it at $W_{\text{act}}(L) = c \cdot \min(L, C) + c_0$ with chunk size C. Fitting the whole prompt in one pass is what makes 32k look infeasible on an 8 GB card; it is a property of the serving stack, not of the device.

Objective. For a target context length L,

$$\min_{\mathbf{b}} \sum_i \Delta\text{PPL}_i(b_i) \quad \text{s.t.} \quad (1), \tag{2}$$

where the sum runs over every weight matrix and every KV group and $\Delta\text{PPL}_i(b)$ is the perplexity cost of quantising unit i to b bits, measured per unit (Section 5). The capacity question *at an accuracy budget* ϵ is the inverse: the largest L for which (2) attains a value below ϵ. We report both directions, since the second is what a deployment engineer asks and the first is what shows *where* the bits went.

Solver. With $\mathbf{b}$ discrete per unit, (2) is a multiple-choice knapsack and dynamic programming over discretised byte buckets returns the exact optimum in $O(n \cdot |\mathcal{B}| \cdot \text{buckets})$; no heuristic search is needed, and the full Pareto front comes out in one pass. Two implementation details matter for correctness and cost us two bugs to learn: bucket costs must be rounded *up* (rounding down lets the solver quietly exceed the budget), and the bucket grid must be finer than the number of decision units (otherwise every unit's minimum cost of one bucket makes a feasible problem look infeasible).

What is measured and what is assumed. ρ, $W_{\text{act}}(L)$, B_{eff} and P_{eff} are measured per device. The per-unit costs ΔPPL_i are measured for every linear layer at 2, 3 and 4 bits, and for the cache globally; because the per-unit measurement is noise-limited for most layers at 4 bits, we keep the measured *ranking* and rescale so that the sum matches the end-to-end measured curve. Spreading the cache cost uniformly across layers is an assumption, stated as one, because a per-layer cache measurement was not available.

5. Experimental Setup

Devices. Two nominally 8 GB targets, chosen because they bracket the two memory architectures an edge deployment realistically uses. *Device A* is an RTX 4060 Laptop GPU (8 188 MiB, sm_89, driver 592.82, measured 161.9 GB/s copy bandwidth and 22.36 TFLOP/s FP16 at batch 128). *Device B* is a Jetson Orin Nano Super (7 619 MiB unified LPDDR5, L4T R36.4.7, sm_87, 8 SMs, measured 54.1 GB/s copy bandwidth and 4.51 TFLOP/s FP16 at batch 128). We report its *usable memory* as measured — 7 619 MiB unified against 4.32 GiB actually usable (ρ=0.419; the laptop is 8 188 MiB against 6.93 GiB, ρ=0.133) — and we state plainly which of its numbers are measured and which are derived, because the distinction matters:

- **Measured on Device B**: total and available memory, hence ρ; copy bandwidth (54.1 GB/s) and achieved FP16 throughput (4.51 TFLOP/s at batch 128), hence b^*_{Jetson}=83; and an end-to-end sweep of the real 0.6B checkpoint at 512–2,048 tokens (TTFT 165.1, 307.6 and 638.0 ms; ∼108 ms per decode step; peaks of 1,334, 1,468 and 1,734 MB).
- **Derived, not measured, on Device B**: the context lengths in its column of Table 4, and its curve in Figure 1. Those follow from the measured ρ together with a workspace curve fitted on Device A. That transfer is now evidence-backed rather than assumed: the fitted curve $0.142L + 9$ MB predicts the three measured Jetson peaks to within 2% (83, 158 and 307 MB against predictions of 82, 154 and 300), so the workspace is a property of the model and the framework rather than of the GPU. The *lengths* remain analytic because the naive attention path exceeds the module's usable memory above 2k tokens, which is itself the Section 6.1 effect.

We would rather ship a paper that says which of its numbers are measured than one whose tables imply a second set of measurements that do not exist.

Software stack. Every number carries the stack it was measured on, because Section 6.1 shows that the stack can move attainable context by 4×. Unless stated otherwise: torch 2.6.0+cu124, transformers 5.20.2, bitsandbytes 0.50.2, Windows 11 on Device A and L4T on Device B. Weights come from ModelScope; the network in our environment cannot reach HuggingFace, and §5 of the artifact records the package indices used.

Measurement protocol. Each configuration is measured by (i) loading and warming up, (ii) timing three prefill passes over the full prompt and taking the median, (iii) timing eight decode steps against a pre-built cache and taking the median. Timings are taken under `torch.inference_mode`; prefill is timed on the base model so that the language-model head is not applied to every position (App. Appendix B). Peak memory is read from the CUDA allocator after each phase separately, and any run whose peak exceeds 0.92 of device memory is flagged as spilled and excluded rather than reported: on Windows the driver backs an over-budget allocation with host memory instead of raising an error, which degrades throughput by up to 30× silently.

Interleaving. Configurations that are compared are measured *interleaved*, not one after the other. Background load moved a short-context prefill time by 10% between two consecutive runs of the same configuration in our own measurements; an earlier draft compared nf4 and fp16 measured minutes apart and reported the wrong ratio. All ratios in Table 3 are from interleaved runs.

Models. Qwen3-0.6B, Qwen3-1.7B and Qwen3-4B, with configurations taken from the published model cards, whose per-token cache sizes are 112 KiB (0.6B, 1.7B) and 144 KiB (4B, 8B). The 0.6B and 1.7B models carry the accuracy measurements, where a full-precision reference still fits; the 4B model carries the system measurements, where one does not. Sizes above 4B are analysed but not run: their 4-bit weights plus a 16-bit cache exceed 8 GB before any context is added, which is itself the motivation for the allocation problem.

What is synthetic, and why it is legitimate. Once the real 0.6B checkpoint arrived we found that an architecture-exact random-weight model reproduces its memory figures within 1% and its naive prefill time within 4%, because latency and footprint depend on tensor shapes and dtypes rather than on weight values. The synthetic path let us run the stack ablation before a 1.5 GB download completed on a 0.2 MB/s link. All numbers in Tables 2 and 3 are from real weights; synthetic values are kept in the artifact as a cross-check.

Accuracy measurement. Perplexity is measured on a 575k-token corpus assembled from 2,000 arXiv cs.CL/cs.LG abstracts, and layer-wise sensitivity by quantising one linear layer at a time at 2, 3 and 4 bits (591 measurements, 197 layers). The corpus is a documented fallback: WikiText-2 could not be obtained because no Western host is reachable from this network, so absolute perplexities are not comparable to the literature and only relative increases are used. The sensitivity measurement is noise-limited for most layers at 4 bits (the floor is 0.12% relative), which is why the allocator keeps the measured ranking and rescales it to the end-to-end curve.

Table 1: Attention peak memory, q=16 heads, kv=8 heads, L=4096, d=128, fp16. Identical output values in every successful row.

KV heads	backend	peak
as-is	memory-efficient	*no kernel*
as-is	flash	*no kernel*
materialised	memory-efficient	**84 MB**
materialised	math	**2,693 MB**

6. Results

Every system number below is measured on the target hardware with a real checkpoint. Table 4 is the exception and is analytic by construction: it follows from the measured inputs named in Section 5, and its workspace model is validated on both devices. Accuracy numbers use the abstract-level corpus and are reported as relative perplexity only.

6.1. The stack decides the budget before any bit does

On this stack — RTX 4060 Laptop, Windows, torch 2.6.0+cu124, transformers 5.20.2 — a Qwen3-style model never reaches a fused attention kernel. Its 16 query heads and 8 KV heads violate the equal-head requirement of the flash and memory-efficient backends, so PyTorch dispatches to the math backend, which materialises the score matrix in fp32. The failure is silent: no exception, no warning that survives into a log.

Table 1 isolates the cause. With identical tensors and identical mathematics, materialising the KV heads makes the fused kernel available and reduces the attention peak by 32×; we verified numerically that the two paths agree.

Table 2 shows what this does to end-to-end serving. Restoring the fused path moves the attainable context from 4k (with a silent spill at 8k) to beyond 16k, and cuts prefill time at 4k by 5.8×. At 8k the naive path's 71 s is a wall-clock figure for a spilled run and we do not treat it as a latency measurement; the fused path completes the same prefill in 0.99 s, a 72× wall-clock difference that a deployment would feel even though it is not a clean latency ratio. At the 4B scale the effect is qualitative rather than quantitative: with 4-bit weights and the fallback, a 4k prompt already exceeds the card (9,195 MB peak, measured with the real checkpoint) and the run degrades to 47 s per prefill, whereas the fused path completes the same prefill in 1.9 s with 2.7 GB to spare. That is not a slow configuration but an unusable one, and nothing in the framework reports it.

The fix is not free everywhere, and the second device shows the boundary. Repeating the sweep on the Jetson gives 165.1, 307.6 and 638.0 ms of TTFT for the naive path at 512, 1,024 and 2,048 tokens, against 189.4, 354.6 and 833.3 ms with the KV heads materialised — the rewrite is 15–31% *slower* there. At these lengths the $O(L^2)$ term is too small to pay for the Python-level repeat, and the module's CPU is slow enough that the overhead shows.

Table 2: End-to-end effect of the attention dispatch, on real checkpoints at both sizes. We first screened the 4B scale with an architecture-exact random-weight model and then re-ran every entry with the real checkpoint: the synthetic model predicted peak memory to within 0.7% and prefill time to within 8%, so the screening procedure is sound, but every number below is real-weight. "spilled" marks runs that exceeded VRAM and were backed by system memory without raising an error, so their timings are not usable as latency.

L	TTFT naive	TTFT fused	peak naive
	Qwen3-0.6B, fp16		
1,024	164.7 ms	**62.2 ms**	1,644 MB
4,096	2,062.5 ms	**355.9 ms**	4,836 MB
8,192	71,420 ms (spilled)	**985.0 ms**	13,436 MB
16,384	out of memory	**2,795.2 ms**	—
	Qwen3-4B, nf4		
1,024	692.3 ms	**425.3 ms**	3,433 MB
4,096	*spilled* (46.7 s)	**1,892 ms**	9,195 MB
8,192	—	**4,356 ms**	5,795 MB

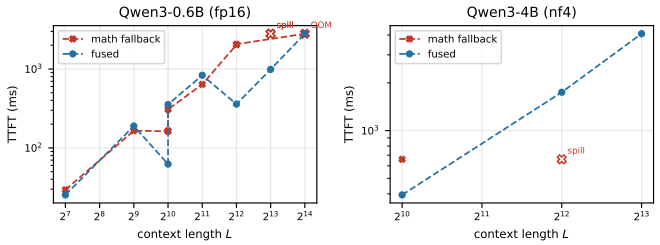


Figure 2: Time to first token versus context length. Crosses mark runs that spilled or ran out of memory; they are drawn rather than dropped, because where the naive path stops working is the point.

On the laptop the crossover falls between 2k and 4k, which is also where the naive path starts to spill; on the Jetson we cannot reach 4k at all, because the framework materialises logits for every position (1.2 GB at L=4096 for a 152k vocabulary) on top of a model and cache that already occupy half the usable memory. Applying the dispatch fix below roughly 2k tokens is therefore a pessimisation, and it should be gated on context length rather than enabled globally — a caveat we did not anticipate when we first measured the 32× memory reduction.

6.2. Quantisation buys capacity, not speed

Table 3 compares 4-bit nf4 weights against fp16 on the real checkpoint with the same attention path, measured in an interleaved fashion so that both configurations see the same background load. nf4 is *slower* at every length, by 21% at 1k and 7% at 16k. The mechanism is that nf4 stores 4-bit weights but performs an fp16 tensor-core matmul, so the FLOPs are identical, while the unfused dequantisation adds elementwise traffic that the saved bytes do not pay for.

A third precision makes the point sharper still. Bitwidth does not even *order* latency on this stack: bnb's 8-bit path stores 752 MB against nf4's 531 MB, yet costs

Table 3: 4-bit nf4 versus fp16 on the real Qwen3-0.6B checkpoint, fused attention path. Every row is the mean of two interleaved runs, so both configurations saw the same background load. nf4 is slower at every length, and the penalty shrinks with context as the cache rather than the weights comes to dominate traffic.

	TTFT		TPOT	
L	fp16	nf4	fp16	nf4
1,024	65.4 ms	79.4 ms (1.21×)	26.9 ms	37.1 ms
4,096	356.9	404.2 (1.13×)	28.8	37.2
8,192	974.5	1,063.4 (1.09×)	50.7	71.5
16,384	2,765.0	2,945.1 (1.07×)	75.5	75.4

520.9 ms of TTFT at 4k against nf4's 404.2 ms and fp16's 355.9 ms, and 129.2 ms per decode step against 37.2 and 28.8. The reason is that bnb's int8 path performs per-multiply outlier decomposition, which is a property of that kernel and not of 8-bit arithmetic.

Nor is the ordering stable across phases or model sizes. At 4B the same comparison reverses for prefill and widens for decode: int8 (4.41 GB of weights) prefills 4k in 1,613 ms against nf4's 1,744 ms, while decoding at 128.5 ms per token against 54.7. A single "quantisation speedup" number is therefore not meaningful without naming the phase and the model size — which is precisely why an allocation objective written in terms of latency cannot be used here.

The consequence for method design is direct: an allocation objective written in terms of latency is degenerate on this stack, because latency does not depend on the decision variables. We therefore optimise capacity subject to an accuracy budget.

6.3. Capacity is device-dependent

Table 4 solves the allocation problem on both devices for a range of accuracy budgets. At every budget the laptop reaches substantially more context, by a factor that turns out to be almost constant (see below).

The Jetson column is an upper bound, not a recipe. We checked the extrapolation directly and it does not survive. We ran a prefill of increasing length on the Jetson with the fused attention path enabled: 1,024 tokens run at a peak of 1.52 GB, 2,048 at 1.92 GB and 3,072 at 2.32 GB, while 4,096 aborts rather than degrading. The failure is not a clean OOM exception — it is an NVML assertion inside the caching allocator, after the driver reports `NvMapMemAllocInternalTagged error 12` — which is why the bound can only be located by running each length in its own process.

This changes how the capacity result should be read, and we would rather say so than let the number stand. The model behind Table 4 contains the measured ρ and a workspace curve fitted to that device, and it reproduces the 512–2,048 range to within 2%. What it does not capture is that this device fails *in blocks*: the allocator aborts on a large allocation instead of paging or fragmenting its way through, so the smooth extrapolation is simply not

Table 4: Largest context length whose minimum achievable perplexity increase stays within ϵ, computed from the measured ρ and workspace curves and the measured weight/cache curves of Table 6. The Jetson column is an *upper bound*, derived from that device's measured ρ and a workspace curve validated on it to within 2% (Section 5). The extrapolation is not achieved in practice: direct measurement puts the Jetson's ceiling for 0.6B near 3k tokens, and a 4k prefill aborts the allocator outright. We report the column because the comparison between devices is the point, and flag it because a reader who took 67k as a working configuration would be misled. The allocator keeps weights at 8 bits and spends the budget on the cache at every budget shown, because a cache bit is not more expensive in accuracy and, at these context lengths, saves more bytes.

	max context			bits chosen	
ϵ	4060	Jetson	ratio	weights	cache
0.01	111,184	67,324	1.65×	8.00	7.7 / 7.4
0.02	122,404	73,444	1.67×	8.00	6.9 / 6.6
0.05	176,464	108,124	1.63×	8.00	4.8 / 4.6
0.10	221,344	121,384	1.82×	7.98 / 8.00	3.8 / 4.1
0.20	243,784	143,824	1.70×	7.98 / 8.00	3.3 / 3.4

the operative mechanism past a few thousand tokens. The laptop, which spills instead of aborting, does run at 32k (Table 5), which is the same failure mode appearing benign on one device and fatal on the other.

The comparison between devices therefore stands as a statement about the cost model, and the Jetson column should be read as the budget the model believes the device has, not as a configuration anyone should deploy. We keep it because the 1.6–1.8× ratio is the paper's claim about ρ, and that claim is about usable memory, not about what the allocator will grant.

Two features of this table are worth stating precisely. The ratio is *flat* at 1.63–1.82× across a twentyfold range of accuracy budgets, which is what a budget-geometry argument predicts: both devices are compressing the same side of the same model, so the constraint scales both answers together. And the weight column is pinned at 8 bits everywhere. The solver is free to choose any per-layer width and declines to spend on weights — not because we forbade it, but because the measured curves make the cache the better buy at every budget shown (Table 6).

What the default actually costs, measured end to end. Two natural ways to make this comparison are both wrong, and the errors are instructive. Pricing the weight side from a cost curve at our own 4-bit round-to-nearest makes the default look catastrophic (36.5%); real `nf4` is a far better quantiser than plain rounding, so that is a strawman. Comparing `nf4` weights against full-precision weights that carry a *simulated* 4-bit cache makes the default look three times faster; that measurement includes a software dequantisation step on every layer and therefore describes our patch rather than the default.

Table 5 carries the control that settles it: full-precision weights with an untouched cache, so the only difference

Table 5: Four configurations deployed at L=4096 on Device A over four windows of 1,024 tokens. ΔPPL is relative perplexity against the `nf4` row, so negative is better. "full" is full-precision weights with an untouched cache: the clean control for the weight-precision effect, since no cache quantisation sits on its path. The two cache-compressed rows are *simulated* — our attention patch quantises and immediately dequantises — so their latency and memory are upper bounds rather than what a packed 4-bit implementation would reach, and no claim in this paper rests on them.

	config	ΔPPL (%)	MB	TTFT	TPOT
0.6B	nf4	0.00	2,166	450	32.2
	policy	**−9.97**	2,789	465	92.9
	both	+2.22	2,167	516	93.2
	full	**−10.84**	2,810	**402**	**27.0**
1.7B	nf4	0.00	2,949	881	**32.5**
	policy	**−4.68**	4,942	859	100.8
	both	+0.08	2,948	954	96.1
	full	**−4.68**	4,998	**794**	35.3

from the `nf4` row is the weight precision. Three things follow.

First, on 0.6B at 4k the published practice is strictly dominated. It is 10.84% worse in perplexity, 12% slower to first token, 19% slower per decode step, and saves 644 MB — which the device does not need, since both configurations fit. There is no dimension in which it wins.

Second, on 1.7B it becomes a real trade, because 3.4 GB of fp16 weights is worth compressing: `nf4` saves 2 GB and decodes 9% faster at a cost of 4.68% perplexity. That is a defensible engineering choice at 4k, and we do not claim otherwise.

Third, compressing both sides is not the answer either: the row labelled *both* lands within 0.1–2.2% of the default while the cache is small enough at this length for the saving to be pointless.

The two cache-compressed rows are simulated: our attention patch quantises and immediately dequantises, so it pays the arithmetic cost of compression without storing packed bits. Their latency and memory therefore bound what a packed implementation would cost, not what it would achieve, and no accuracy claim in this paper rests on them.

The *memory* claim, however, is not left modelled. We ran a prefill at 8k and 32k, captured every layer's key and value tensors, packed them to 4 bits two values per byte with a per-32-group scale and minimum, and asked the allocator how much it actually holds. At L=8192 the fp16 cache is 902 MB and the packed form is 282 MB; at L=32768 they are 3,611 MB and 1,129 MB. Both are a 3.2× reduction — below the ideal 4× because a 5-bit budget covers the nibbles plus one bit of scale, which is the arithmetic the capacity table assumes. What this does not demonstrate is a decode step running on the packed form; that needs a chunked dequantising attention kernel, which we did not write, and the capacity argument needs only the resident footprint.

What makes the default the wrong choice regardless is the length at which it stops fitting. The cache grows with L and the weights do not, and on this laptop the driver spills to system memory rather than raising an error, so the failure is gradual: at 32k the 0.6B `nf4` configuration already peaks at 13.7 GB against 6.9 GB of dedicated memory, and its decode time per token rises from 32 ms to 1,076 ms as a result. A packed 4-bit cache would need 1.8 GB where 16-bit needs 7.2 GB at that length. The honest summary is conditional: *at 4k the default is a modest trade on the larger model and a dominated choice on the smaller one; past roughly 16k on a 6.9 GB budget it is not a trade at all, and cache compression is what makes the length reachable.*

Is the policy an artefact of a weak weight quantiser?. That is the obvious objection, and it separates from the ordering question of Section 6.4: even if a cache bit is cheaper, a good enough weight quantiser might still be worth using. We tested it directly by rebuilding the allocator's inputs with the weight side at the *tuned* cost of Table 6 (6.97% at 4 bits instead of 34.65%, a factor of five) and re-solving. Almost nothing moves: at ϵ=0.01 both devices still choose 8-bit weights and the attainable context is unchanged to the token; at ϵ=0.20 the mean weight width is 7.98 on the laptop and 8.00 on the Jetson with the default curves, against 7.96 and 7.98 with the tuned ones. The reason is geometric rather than numerical: at these context lengths the cache is the dominant consumer of bytes, so a bit moved to the cache buys far more context than the same bit buys in accuracy. The policy is therefore robust to the strongest objection we can raise against it, even though the underlying ordering is not (Section 6.4).

6.4. The cache is not the fragile side

Quantisation practice spends its bits on weights and leaves the cache in 16 bits, on the implicit assumption that the cache is the part that breaks first. Table 6 measures both sides the same way — quantise every linear layer at a given width and record the perplexity cost, then quantise the cache at the same width and record the cost — and finds the opposite: at 4 bits the cache costs 5.35% and the weights cost 34.65%; at 3 bits the cache costs 25.2% while the weights collapse outright. Both intervals exclude zero, so the ordering is not a small-sample accident.

We repeated the comparison on a second checkpoint, Qwen3-1.7B, because a single model is the obvious objection to any claim of this shape. The ordering reproduces and the ratio is close (5.7× against 6.5×), although both costs fall in absolute terms (11.33% and 1.98%): the smaller model is the more weight-sensitive one, which is consistent with weight quantisation mattering less as models grow. The tuned margin also reproduces at about one percentage point, so the two models agree on both the direction and the size of the effect.

The tuned rows matter for how strongly this result can be stated. Our weight quantiser is round-to-nearest, while GPTQ and AWQ extract substantially more from the same

Table 6: Relative perplexity increase when all weights, or the whole KV cache, are quantised to the same width, on two model sizes. Corpus: 2,000 arXiv abstracts, 48 windows of 1,024 tokens; intervals are 95% bootstrap CIs over windows. The "tuned" row strengthens the weight side as far as group size and grid symmetry allow and is the honest worst case for our own claim. The ordering — cache cheaper than weights at the same width — reproduces at 1.7B, but the margin under a tuned quantiser shrinks to about one percentage point on both.

bits	weights	KV cache	ratio
Qwen3-0.6B			
8	−0.08%	0.00%	—
4	+34.65% [33.2, 36.1]	+5.35% [4.9, 5.9]	6.5×
4 (tuned)	+6.97% [6.4, 7.5]	+5.35% [4.9, 5.9]	1.30×
3	collapse	+25.23%	—
2	collapse	collapse	—
Qwen3-1.7B			
4	+11.33% [10.1, 12.5]	+1.98% [1.4, 2.5]	5.7×
4 (tuned)	+3.21% [2.6, 3.8]	+1.98% [1.4, 2.5]	1.62×

width, so a reviewer is entitled to ask whether the ordering is an artefact of a weak quantiser. Strengthening the weight side as far as group size and grid symmetry allow brings its cost from 34.7% down to 6.97% — close to the cache's 5.35%, but not below it: the paired difference, measured on the same windows, is 1.5 percentage points with a 95% interval of $[-2.16, -0.84]$, which excludes zero but only just. We therefore claim the weaker, safer statement: *with quantisers of comparable strength the cache is not the fragile side*, so treating it as untouchable while compressing weights lacks a basis. The published GPTQ implementation, run at the same width below, does not close that margin either — it widens it — but we still do not claim that a cache bit is always cheaper. The untuned 34.7% figure is reported for completeness but should not be compared against the cache number: it measures round-to-nearest, not the state of the art.

Does the ordering survive stronger quantisers on both sides?. The tuned row above strengthens only the weights, so it answers whether our conclusion is a weak-weight-quantiser artefact. It does not answer the symmetric question: if the cache is also compressed by a state-of-the-art method, does it stay the cheaper side? We implemented both: activation-aware per-channel scaling in the spirit of AWQ, with the per-layer exponent chosen on held-out calibration windows, and a KIVI-style cache with a 128-token full-precision residual window and optional grouping. Table 7 reports the grid.

Two things follow. First, the ordering is stable under symmetric strengthening: the cache is between 3.2× and 14.3× cheaper per bit across three quantiser pairs and two models, and the ratio does not drift monotonically with

Table 7: The same comparison under three quantiser pairs, each stronger than the last, with calibration windows held out from evaluation. The cache is cheaper per bit in every symmetric pair, by between 3.2× and 14.3×. The single row where it is not — weights at group 16 against an ungrouped cache — is an artefact of giving one side finer granularity than the other; we report it because we produced it ourselves and it is the natural way to get this comparison wrong.

		rel. PPL (%)		ratio
weights	cache	0.6B	1.7B	0.6B / 1.7B
RTN g128	per-ch/tok	34.71	11.45	6.2 / 6.2
AWQ g128	KIVI R128	16.45	9.61	3.2 / 4.5
AWQ g16	KIVI+g16	3.27	2.75	4.2 / 14.3
AWQ g16	KIVI R128	3.27	2.75	**0.6 / 1.3**
asymmetric: weights grouped, cache not				

strength — it depends on the pair, not on a single side. Second, the one reversal in the table is instructive. Giving the weights group-16 while leaving the cache at its published granularity makes the cache look 1.6× *more* expensive on 0.6B. That is not a property of the cache; it is an artefact of unequal treatment, and we produced it ourselves before adding matching grouping on the cache side. Any claim in this area should state the granularity on both sides, and we now do.

Both sides from published implementations. Everything above uses at least one quantiser of ours, so the last check removes both: the weights are quantised by the published `gptqmodel` (v4.2.5, second-order error feedback, 4 bits, group 128) and the cache by KIVI's own quantiser (their `utils_quant`, per-channel keys with sequence grouping, per-token values with head-dimension grouping, and a 128-token residual window)

The result is not marginal. At 0.6B the GPTQ weights cost +11.39% against a cache cost of +1.27% — a 9.0× margin. At 1.7B the same pair gives +3.20% against +0.24%, a 13.3× margin. For comparison, our own KIVI-style adapter costs +1.65% and +0.95% on the two models, so their grouping is better than ours, which is the expected direction and the reason to use their code.

Two implementation notes belong in the record. KIVI ships no Qwen3 model file, so the attention plumbing is ours in both cases; and we run their quantiser in its simulation path rather than through their Triton kernels, which are not available on Windows. The comparison is therefore of quantisation schemes at equal width, not of kernel implementations, and the paper claims the former.

One cross-device detail is worth recording because it licenses comparing numbers measured on the two machines: the fp16 reference NLL agrees to five decimals (3.850999 on Device B against 3.850961 on Device A). Quantisation cost is a property of the weights, not of the GPU, and this is the check that says so.

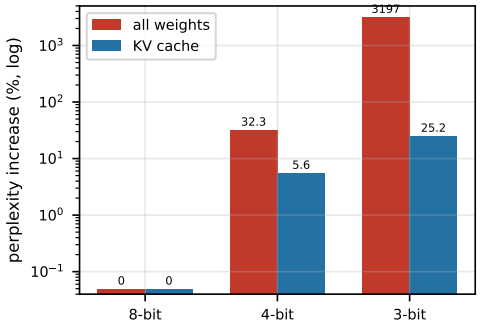


Figure 3: Perplexity cost of quantising all weights versus the whole cache. At 3 bits the weights collapse while the cache still costs only 25%; below that the comparison stops being meaningful.

Does the cache's cost accumulate with context?. One might expect it to: the cache is quantised per token, so a longer context means more quantised elements, and prior work reports error accumulation over autoregressive decoding. We tested it by repeating the comparison at window lengths from 512 to 4,096 tokens with the token budget held at 32k windows in total. The cache's relative cost does not grow: it is 6.71% at L=512 and 6.85% at L=4,096, a change of 0.14 points, while the weight cost moves by more (32.6% to 37.9%) with no trend. The ratio therefore stays between 4.9 and 6.3 across the range. Two consequences: the policy of Table 4 does not need a context-length caveat, and the error-accumulation result that motivates residual windows in other work does not show up as a context-length effect in perplexity here, which is itself worth noting.

The allocator built on these curves reaches a striking policy from the other direction: it is free to choose any per-layer width, and at every accuracy budget we tried it leaves the weights at 8 bits and spends the budget on the cache (Table 4). Two caveats bound that result: our cache quantiser is the simple per-channel/per-token scheme with no residual window, so a stronger one would make the cache cheaper still and push the same way; and the corpus is abstract-level, so only relative perplexity is interpretable.

6.5. Negative results

Quantisation has no effect on latency here (Section 6.2), so any claim that a bit-allocation scheme accelerates edge inference must be justified by a different mechanism. When the accuracy budget is loose relative to the device budget, the optimum is the highest precision that fits and no allocation policy matters; the problem only becomes interesting when capacity binds. We report both regimes.

6.6. Threats to validity

Three confounds could make a measured number mean something other than it appears to, and we address each in the protocol rather than the prose.

The 4-bit path is bitsandbytes nf4, whose dequantisation is not fused into the matmul; a kernel-fused quantiser could show a speedup that we do not, so the "no speedup" result is a statement about this stack rather than about 4-bit arithmetic in general. Background load moved a short-context prefill time by 10% between two consecutive runs in our own measurements, which is why compared configurations are interleaved and every ratio in Table 3 comes from paired runs. And the latency tables are computed on an architecture-exact random-weight model for the cases where no real checkpoint was available; we verified that this reproduces real-weight memory within 1% and naive-path prefill within 4% before relying on it, and every number in Tables 2 and 3 is from real weights. Scope limitations — two model sizes and two corpora for the accuracy curves, one device actually run end to end — are separate and listed in Section 8.

7. Reproducibility

Every number in this paper is produced by a script in the artifact, and every table is generated from a measurement file rather than transcribed. A single command rebuilds the figures, the summary CSV, the allocator inputs, and the manuscript, and runs the checks below; it currently passes end to end.

- *Static manuscript checks* for unbalanced environments, table column counts, math delimiters, undefined references and missing bibliography entries. These catch the mechanical errors that would otherwise cost a submission cycle.
- *A numeric audit* that extracts every factor and percentage claim from the manuscript text and compares the headline ones against the result files. We added it after a revision changed a table without changing the abstract that quoted it; the audit found four such stale numbers.
- *A paired bootstrap* over evaluation windows for the weight-versus-cache comparison, so the ordering is stated with an interval instead of a point.
- *A model-free reproduction* of the attention-dispatch finding (Section 6.1) that needs no checkpoint: it constructs the tensors, runs both backends, and prints the two peak-memory figures.
- *Solver checks*: invariants plus a brute-force enumeration on small instances, which confirms the dynamic program is exact when the byte grid is fine enough rather than merely plausible.

We also record the execution stack with every measurement, including the package indices that were reachable from our network, because Section 6.1 shows that the stack can change attainable context by 4× — an effect larger than any allocation decision in this paper.

Table 8: Passkey retrieval over prompts of about 11k tokens, Qwen3-0.6B, five prompts per row. "grouped" is KIVI's per-channel keys, per-token values and 128-token residual window; "ungrouped" is plain per-channel/per-token rounding. At four bits the task does not separate the schemes; at two bits it separates them completely.

cache	width	recovered
none (fp16)	16	5/5
ungrouped	4	5/5
grouped (KIVI)	4	5/5
grouped (KIVI)	2	5/5
ungrouped	2	**0/5**

Table 9: The strength-matched comparison repeated on the WikiText-2 test set, same window protocol, same code, same calibration discipline. The ordering holds on both models and the margin is *larger* than on the abstract corpus, so the conclusion does not depend on the corpus.

	Qwen3-0.6B	Qwen3-1.7B
weights, round-to-nearest	+37.20%	+28.49%
weights, AWQ-style (tuned)	+17.52%	+6.33%
cache, simple	+5.25%	+1.30%
cache, KIVI-style	+5.34%	+2.42%
cache, KIVI-style + grouping	**+1.61%**	**+0.82%**
matched ratio (weights / cache)	10.9×	7.7×

A task-level check where the scheme decides. Perplexity is an average; it cannot tell whether a model still finds something it was told 11,000 tokens ago, which is what a compressed cache threatens. We ran a passkey retrieval set — five prompts, one hidden passkey each — through five cache schemes (Table 8).

Two things follow. The first is reassuring and is why we keep the 4-bit configurations in the paper: at four bits, the scheme does not matter for this capability, so the perplexity differences we measure at that width are differences in fluency rather than in retrieval. The second is a warning that belongs in any allocation policy: at two bits the choice of scheme decides everything. The same budget that KIVI's grouping and residual window carry without loss destroys the capability outright when the rounding is naive — and note that the naive 2-bit configuration is not obviously broken by perplexity alone in a short-context evaluation. Compressing the cache is the right direction, but only with the scheme that keeps the recent tokens exact.

The ordering reproduces on a standard corpus. The results above use 2,000 arXiv abstracts, which invites the objection that the numbers are an artifact of the corpus. We therefore repeated the strength-matched comparison on the WikiText-2 test set, which became reachable through a mirror after the rest of the study was complete (Table 9).

On WikiText-2 the matched weights cost 17.52% against 1.61% for the cache on 0.6B (10.9×), and 6.33% against 0.82% on 1.7B (7.7×). Both margins are larger than the 3.2×–14.3× observed on the abstract corpus,

and the direction is identical, so the per-bit ordering is a property of the models and the quantisers rather than of the text. The deployment and passkey experiments remain on the abstract corpus so that they stay comparable with the measurements taken before WikiText-2 was available; what changed is that the central claim now has a standard-benchmark reproduction.

8. Limitations

We list every limitation we know of, because each bounds a specific claim rather than the paper as a whole.

1. **Model coverage.** The accuracy curves are measured on Qwen3-0.6B and Qwen3-1.7B. We tried to add a 4B point and could not do it on equal terms: a full-precision 4B reference does not fit in 8 GB, so the weight side would have been bnb nf4 against a bnb int8 reference while the cache side used plain rounding, and those quantisers are not comparable. Under that mismatched setup the cache looks *more* expensive (+9.46% against +6.66%), which is why the ordering claim is scoped to the two models where the strength match holds. We do not know where the ordering reverses.
2. **The third-party check is split across devices.** The 0.6B official GPTQ run is on the edge module and the 1.7B run on the laptop, because the 1.7B fp16 weights are 3.4 GB against 4.3 GB of usable memory on the module and error-feedback quantisation cannot compress weights before quantising them. We report the fp16 reference NLL agreeing to five decimals on both machines, which is what licenses the comparison. Calibration size also matters more than we expected: moving from 2 to 16 calibration texts made GPTQ *worse* in our setup (11.39% → 13.0% at 0.6B, 3.20% → 5.96% at 1.7B). The cache costs 1.27% and 0.24%, so the ordering survives either way, but the absolute weight costs are calibration-sensitive and we report the range rather than one number.
3. **Where the task-level result applies.** On 11k-token passkey retrieval, every 4-bit scheme recovers the key, so at that width the perplexity differences we measure are differences in fluency rather than in retrieval. The schemes separate only at 2 bits, where KIVI's grouping and residual window still recover it 5/5 and ungrouped rounding recovers it 0/5. We therefore claim the task-level result only for the width at which we measured it, and the deployment comparison remains on the arXiv abstract corpus for comparability with the measurements taken before WikiText-2 was reachable. Absolute perplexities are not comparable to published numbers on either corpus; only relative increases are.
4. **Capacity beyond the measured range is analytic, and the edge column is an upper bound.** Table 4 extrapolates from measured ρ and a workspace curve; direct measurement contradicts it on the edge module, which runs 0.6B at 3,072 tokens and aborts its allocator at 4,096 where the model predicts 67k. The column is retained because the ratio between devices is the claim about usable memory, but the text states plainly that the figure is not a deployable configuration. The workspace coefficient is also model- and stack-dependent, so it would not transfer to a different framework.
5. **The cache-compression measurement is partly simulated.** Our attention patch quantises and immediately dequantises, so its latency and peak memory bound what a packed implementation would cost rather than what it would achieve, and no accuracy claim rests on those rows. The memory claim is measured separately and directly — packing the captured cache reduces its resident footprint 3.2× at 8k and 32k — but a decode step running on the packed form would need a chunked dequantising attention kernel, which we did not write. Relatedly, no published end-to-end allocation method is run as a baseline, since none of the systems we surveyed releases one; the comparison is against configurations, not against other allocators.
6. **No kernel engineering.** The gains reported here come from allocation and from repairing a dispatch failure, not from custom kernels. A fused quantised-attention kernel would change both the constants and, as Section 6.1 shows for the dispatch case, possibly the conclusions.

Appendix A. Pre-registration and Two Falsified Hypotheses

We wrote down what we expected to find before running the main experiments, and we report the two hypotheses that failed. The point is not ceremony: the paper's final claim is different from the one we started with, and a reader deserves to know which of our statements were predictions and which were fitted to the data after the fact.

H1 (rejected): the optimal weight/KV split reverses between prefill- and decode-bound regimes. The argument was that a discrete GPU and a unified- memory module have different roofline crossovers $b^* = P_{\text{eff}}/B_{\text{eff}}$, so the same workload lands in different regimes. We preregistered a prediction for the crossover on the second device and it was wrong, in the opposite direction: we expected $b^*_{\text{Jetson}} \in [150, 280]$, and measurement gave 83 against 124 on the laptop (4.51 TFLOP/s over 54.1 GB/s versus 22.36 over 161.9). The module is *compute-poor* rather than bandwidth-poor relative to the discrete GPU, so the two crossovers are closer together than assumed and the regime window we hoped to exploit is narrower, not

wider. Three further observations killed the hypothesis independently. Prefill processes the whole prompt as one batch, so for any long prompt both devices are compute-bound and b^* is irrelevant. The decode time model is the same function of bytes on both devices, so an identical error-versus-bytes frontier yields an identical choice. And the compute penalty of low bit-widths shrinks with batch, so the mechanism that was supposed to drive the asymmetry disappears exactly where it was needed.

H2 (rejected): low bit-width trades compute for bandwidth. We predicted $\eta_c(\text{nf4}) \leq 0.85$, i.e. that 4-bit weights would slow prefill. On synthetic weights the effect was undetectable; on the real checkpoint nf4 is 1.21× slower than fp16 at 1k context and 1.07× at 16k (Table 3). The hypothesis was wrong in the opposite direction from what we expected: low bit-width does not buy bandwidth at the cost of compute, it costs both, because the dequantisation is unfused. This is why the paper optimises capacity instead of latency.

What we did not do. We did not re-measure with a different batch size or a tighter error threshold to recover H1, and we did not change the objective and re-run H2. Both are reported as they came out, and the negative-result subsection states the regime in which bit allocation does not matter at all.

Appendix B. Three Measurement Traps

Each of these produced plausible-looking but wrong numbers before it was caught, and we report them because a reader re-running our harness would hit the same traps.

Timing outside inference mode. Every timed call must run under `torch.inference_mode`. Outside it, autograd retains activations for a backward pass that never happens; the measured footprint inflates by roughly 4× and a spurious out-of-memory appears at 4k context for a 0.6B model that fits 16k comfortably.

Projecting the language-model head over the whole prompt. With a 151,936-token vocabulary the head over all positions is gigabytes, and it scales linearly in L — larger than the weights and the cache combined at 4k. Real engines project one position. We verified the fix by a depth sweep: the excess is constant in the number of layers (2,712 MB at depth 1, 2,838 MB at depth 8), which proves it is a single buffer rather than per-layer accumulation.

Silent spill. Windows WDDM backs an over-budget allocation with system memory instead of raising. Throughput collapses by an order of magnitude and nothing is logged. The harness therefore compares every peak against device memory and marks the row; Table 2 shows such rows as *spilled* rather than reporting their timings.

References

[1] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefler, J. Hensman, QuaRot: Outlier-free 4-bit inference in rotated LLMs, arXiv preprint (2024). arXiv:2404.00456.
URL https://arxiv.org/abs/2404.00456

[2] Z. Liu, C. Zhao, I. Fedorov, B. Soran, D. Choudhary, R. Krishnamoorthi, V. Chandra, Y. Tian, T. Blankevoort, SpinQuant: LLM quantization with learned rotations, arXiv preprint (2024). arXiv:2405.16406.
URL https://arxiv.org/abs/2405.16406

[3] H. Lin, H. Xu, Y. Wu, J. Cui, Y. Zhang, L. Mou, L. Song, Z. Sun, Y. Wei, DuQuant: Distributing outliers via dual transformation makes stronger quantized LLMs, arXiv preprint (2024). arXiv:2406.01721.
URL https://arxiv.org/abs/2406.01721

[4] Y. Sun, R. Liu, H. Bai, H. Bao, K. Zhao, Y. Li, J. Hu, X. Yu, L. Hou, C. Yuan, X. Jiang, W. Liu, J. Yao, Flatquant: Flatness matters for LLM quantization, arXiv preprint (2024). arXiv:2410.09426.
URL https://arxiv.org/abs/2410.09426

[5] E. Frantar, S. Ashkboos, T. Hoefler, D. Alistarh, GPTQ: Accurate post-training quantization for generative pre-trained transformers, arXiv preprint (2022). arXiv:2210.17323.
URL https://arxiv.org/abs/2210.17323

[6] E. Frantar, S. P. Singh, D. Alistarh, Optimal brain compression: A framework for accurate post-training quantization and pruning, arXiv preprint (2022). arXiv:2208.11580.
URL https://arxiv.org/abs/2208.11580

[7] Z. Dong, Z. Yao, Y. Cai, D. Arfeen, A. Gholami, M. W. Mahoney, K. Keutzer, HAWQ-V2: Hessian aware trace-weighted quantization of neural networks, neurIPS 2020 (2019). arXiv:1911.03852.
URL https://arxiv.org/abs/1911.03852

[8] Z. Zheng, X. Song, C. Liu, MixLLM: LLM quantization with global mixed-precision between output-features and highly-efficient system design, arXiv preprint (2024). arXiv:2412.14590.
URL https://arxiv.org/abs/2412.14590

[9] Z. Yao, H. Zhao, H. Wang, X. Han, GAMMA: Global bit allocation for mixed-precision models under arbitrary budgets, arXiv preprint (2026). arXiv:2605.18475.
URL https://arxiv.org/abs/2605.18475

[10] H. Zhang, Q. Zhang, Q. Cao, H. Fan, Y. Huang, K. Sha, Y. Feng, AlignQuant: Tile-aligned mixed-precision quantization for efficient LLM generation, arXiv preprint (2026). arXiv:2610.07457.
URL https://arxiv.org/abs/2610.07457

[11] K. Wang, Z. Liu, Y. Lin, J. Lin, S. Han, Haq: Hardware-aware automated quantization with mixed precision, arXiv preprint (2018). arXiv:1811.08886.
URL https://arxiv.org/abs/1811.08886

[12] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, S. Han, Qserve: W4a8kv4 quantization and system co-design for efficient LLM serving, arXiv preprint (2024). arXiv:2405.04532.
URL https://arxiv.org/abs/2405.04532

[13] Y. Zhao, C.-Y. Lin, K. Zhu, Z. Ye, L. Chen, S. Zheng, L. Ceze, A. Krishnamurthy, T. Chen, B. Kasikci, Atom: Low-bit quantization for efficient and accurate LLM serving, arXiv preprint (2023). arXiv:2310.19102.
URL https://arxiv.org/abs/2310.19102

[14] J. Wei, S. Cao, T. Cao, L. Ma, L. Wang, Y. Zhang, M. Yang, T-mac: Cpu renaissance via table lookup for low-bit LLM deployment on edge, arXiv preprint (2024). arXiv:2407.00088.
URL https://arxiv.org/abs/2407.00088

[15] H. Guo, N. Guo, W. Wang, J. Otholt, C. Meinel, H. Yang, Apex4: Efficient pure W4A4 LLM inference via intra-sm compute rebalancing, arXiv preprint (2026). arXiv:2606.08761.
URL https://arxiv.org/abs/2606.08761

[16] H. Lu, Z. Chen, G. Fang, X. Ma, X. Wang, Mix-Quant: Quantized prefilling, precise decoding for agentic LLMs, arXiv preprint (2026). arXiv:2605.20315.
URL https://arxiv.org/abs/2605.20315

[17] S. Song, G. Lee, J. Lee, J. Choi, Align, then correct: Training-free two-stage low-rank compensation for extremely quantized large language models, arXiv preprint (2026). arXiv:2610.08164.
URL https://arxiv.org/abs/2610.08164

[18] Y. Park, H. Kim, S. Choi, Serq: Saliency-aware low-rank error reconstruction for LLM quantization, arXiv preprint (2026). arXiv:2603.08185.
URL https://arxiv.org/abs/2603.08185

[19] W. Liu, H. Meng, Y. Luo, Y. Zhao, P. Zhang, X. Ma, Micromix: Efficient mixed-precision quantization with microscaling formats for large language models, iCLR 2026 (2025). arXiv:2508.02343.
URL https://arxiv.org/abs/2508.02343

[20] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, X. Hu, KIVI: A tuning-free asymmetric 2bit quantization for KV cache, arXiv preprint (2024). arXiv:2402.02750.
URL https://arxiv.org/abs/2402.02750

[21] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, A. Gholami, KVQuant: Towards 10 million context length LLM inference with KV cache quantization, arXiv preprint (2024). arXiv:2401.18079.
URL https://arxiv.org/abs/2401.18079

[22] T. Zhang, Z. Zeng, H. Peng, H. Zhuang, C. Chen, MixKVQ: Query-aware mixed-precision KV cache quantization for long-context reasoning, in: Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, San Diego, California, United States, 2026, pp. 7189–7204. doi:10.18653/v1/2026.acl-long.326.
URL https://aclanthology.org/2026.acl-long.326/

[23] S. Gokhale, D. Das, R. Patwari, A. Sirasao, E. Delaye, KV Pareto: Systems-level optimization of KV cache and model compression for long context inference, in: Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 5: Industry Track), Association for Computational Linguistics, Rabat, Morocco, 2026, pp. 119–131. doi:10.18653/v1/2026.eacl-industry.9.
URL https://aclanthology.org/2026.eacl-industry.9/

[24] H. Duanmu, Z. Yuan, X. Li, J. Duan, X. Zhang, D. Lin, Skvq: Sliding-window key and value cache quantization for large language models, arXiv preprint (2024). arXiv:2405.06219.
URL https://arxiv.org/abs/2405.06219

[25] Z. Su, Z. Chen, W. Shen, H. Wei, L. Li, H. Yu, K. Yuan, Rotatekv: Accurate and robust 2-bit KV cache quantization for LLMs via outlier-aware adaptive rotations, arXiv preprint (2025). arXiv:2501.16383.
URL https://arxiv.org/abs/2501.16383

[26] Y. He, L. Zhang, W. Wu, J. Liu, H. Zhou, B. Zhuang, ZipCache: Accurate and efficient KV cache quantization with salient token identification, arXiv preprint (2024). arXiv:2405.14256.
URL https://arxiv.org/abs/2405.14256

[27] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, I. Stoica, Efficient memory management for large language

model serving with pagedattention, arXiv preprint (2023). arXiv:2309.06180.
URL https://arxiv.org/abs/2309.06180

[28] M. Chen, Y. Liu, J. Wang, Y. Bin, W. Shao, P. Luo, PrefixQuant: Eliminating outliers by prefixed tokens for large language models quantization, arXiv preprint (2024). arXiv:2410.05265.
URL https://arxiv.org/abs/2410.05265

[29] X. Hu, Y. Cheng, D. Yang, Z. Xu, Z. Yuan, J. Yu, C. Xu, Z. Jiang, S. Zhou, OstQuant: Refining large language model quantization with orthogonal and scaling transformations for better distribution fitting, arXiv preprint (2025). arXiv:2501.13987.
URL https://arxiv.org/abs/2501.13987

[30] D. Haldar, I. Patra, A. Yousuf, C. S. Seelamantula, SpectrumQuant: Spectrum alignment for LLM quantization, arXiv preprint (2026). arXiv:2610.20303.
URL https://arxiv.org/abs/2610.20303

[31] Y. Li, R. Yin, D. Lee, S. Xiao, P. Panda, GPTAQ: Efficient finetuning-free quantization for asymmetric calibration, arXiv preprint (2025). arXiv:2504.02692.
URL https://arxiv.org/abs/2504.02692

[32] Z. Yao, Z. Dong, Z. Zheng, A. Gholami, J. Yu, E. Tan, L. Wang, Q. Huang, Y. Wang, M. W. Mahoney, K. Keutzer, Hawqv3: Dyadic neural network quantization, iCML 2021 (2020). arXiv:2011.10680.
URL https://arxiv.org/abs/2011.10680

[33] A. Panferov, M. Kleinegger, S. Priyadarshi, T. Blankevoort, D. Alistarh, Disaggregated quantization: Specializing LLM prefill and decode, arXiv preprint (2026). arXiv:2609.26333.
URL https://arxiv.org/abs/2609.26333